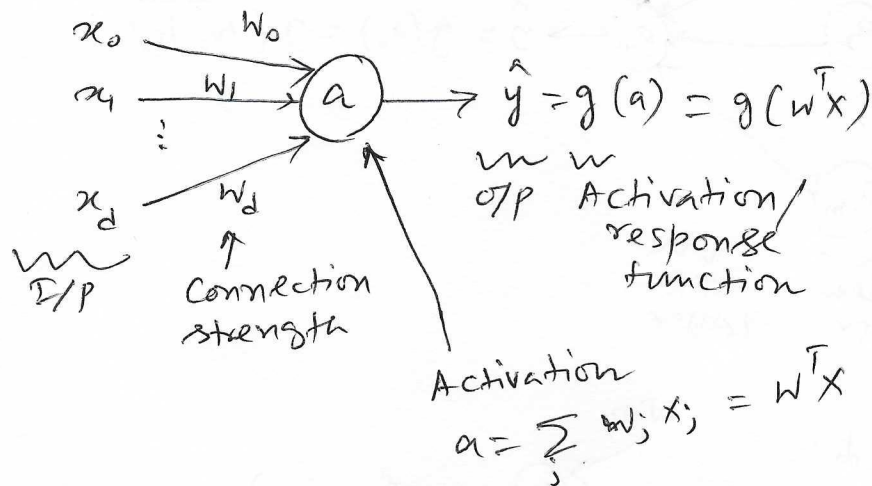


Artificial Neural Network (ANN)

Motivation: mimic human intelligence

A biological neuron looks like a computational unit - Can we model this?



Different types of activation functions:

a) Linear: $g(a) = a$

$\hat{y} = g(w^T x) \Rightarrow$ Linear regression

b) Sigmoid/logistic: $g(a) = \frac{1}{1 + e^{-a}}$

$$\hat{y} = \frac{1}{1 + e^{-w^T x}}$$

c) Step fn: $g(a) = \begin{cases} 1, & a \geq 0 \\ 0, & \text{else} \end{cases}$

$$\hat{y} = \begin{cases} 1 & \text{if } w^T x \geq 0 \\ 0 & \text{if } w^T x < 0 \end{cases} \quad \text{Binary classifier (perceptron)}$$

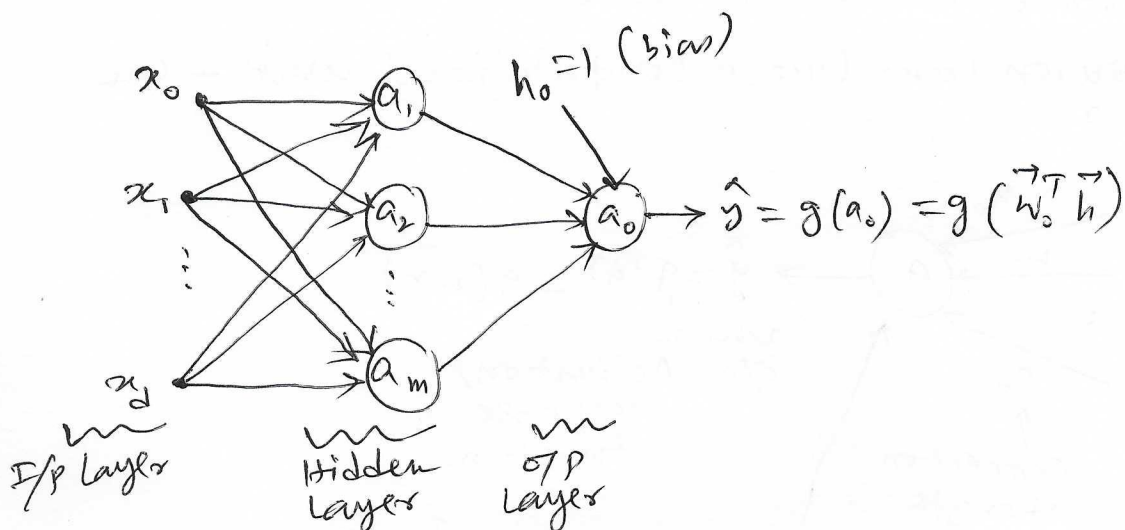
But still this is not adding anything new to whatever we have learned so far.

↓ Solution

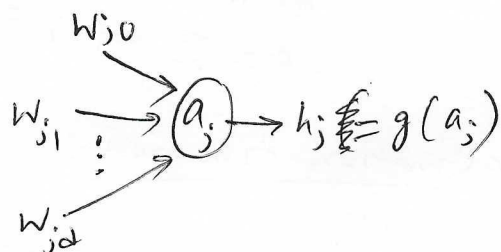
Use multiple linear models!

Hidden layer.

1-hidden layer (3-layer NN)



One single node:



Convention: $w_{\text{destination, source}}$
 $w_{00} \rightarrow \text{bias}$

$$\vec{h} = \begin{bmatrix} h_0=1 \\ h_1 \\ \vdots \\ h_m \end{bmatrix}$$

$$W_0 = [w_{00} \ w_{01} \ \dots \ w_{0m}]$$

$$W_i = [w_{i0} \ w_{i1} \ \dots \ w_{im}]$$

Output activation $a_{\text{out}} = w_{00}h_0 + w_{01}h_1 + \dots + w_{0m}h_m$

$$= \sum_{i=0}^m w_{0i}h_i$$

So we are creating function of functions of functions ...
 Which is highly nonlinear.

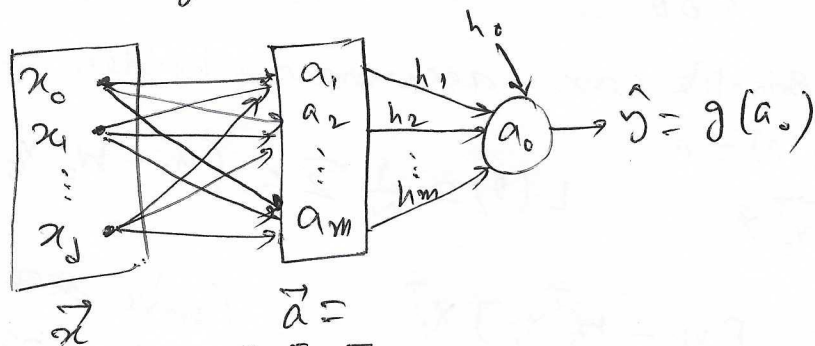
We can also view NN as ~~also~~ a feature extractor (learnable).

$$x \rightarrow \phi(x)$$

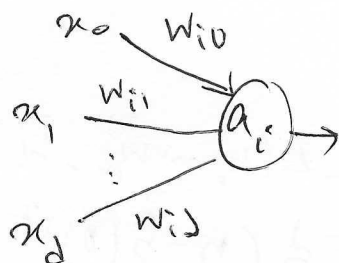
(21)

Forward Propagation (forward pass)

Making predictions from a network



$$\vec{a} = \begin{bmatrix} w_1^T x \\ \vdots \\ w_m^T x \end{bmatrix} = W_h \vec{x}, \text{ where } W_h = \begin{bmatrix} -w_1^T - \\ \vdots \\ -w_m^T - \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \dots & w_{md} \end{bmatrix}$$



$$\vec{w}_i = \begin{bmatrix} w_{i0} \\ \vdots \\ w_{id} \end{bmatrix}$$

$h_j = g(a_j)$ in vector form!

$$\vec{h} = g(\vec{a}) = \begin{bmatrix} h_0 = 1 \\ g(a_1) \\ \vdots \\ g(a_m) \end{bmatrix}$$

$$a_0 = w_{00} h_0 + w_{01} h_1 + \dots + w_{0m} h_m$$

$$W_0 = [w_{00} \ w_{01} \ \dots \ w_{0m}]$$

$$\text{So, } a_0 = \vec{W}_0 \vec{h}, \quad \hat{y} = g(a_0)$$

Q We perform forward pass & get some output. Loss is defined as $L(y^{gt}, \hat{y})$. Compute the gradient of the loss with respect to all the parameters & take a step towards that direction. Computing the gradient is the back-propagation.

Gradient descent

$$\theta^{t+1} = \theta^t - \eta \frac{\partial L}{\partial \theta}$$

Let's start with a simple case - assume no hidden units.

$$g(a) = a$$

$$\hat{y} = \vec{w}_0^T \vec{x}$$

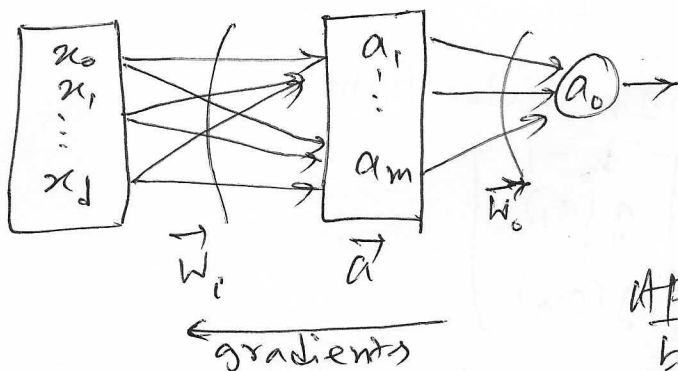
$$L(\theta) = \frac{1}{N} \sum_i \frac{1}{2} (y_i - \vec{w}_0^T \vec{x}_i)^2$$

$$\frac{\partial L}{\partial \vec{w}_0} = \frac{1}{N} \sum_i - \underbrace{[y_i - \vec{w}_0^T \vec{x}_i]}_{\text{error}} \underbrace{\vec{x}_i^T}_{\text{feature}}$$

(we've seen this before in logistic regression)

Now let's bring back the hidden layer. Only change: replace $\vec{x}$ by $\vec{h}$:

$$\frac{\partial L}{\partial \vec{w}_0} = \frac{1}{N} \sum_i - (y_i - \vec{w}_0^T \vec{h}_i) \cdot \vec{h}^T$$



$$L(\theta) = L(\vec{w}_1, \dots, \vec{w}_m, \vec{w}_0) = \frac{1}{2} (y - \hat{y}(\theta))^2$$

Goal: compute $\frac{\partial L}{\partial \vec{w}_0}$, $\frac{\partial L}{\partial \vec{w}_i}$

Approach: Compute gradients backwards w.r.t. $a_0, \vec{w}_0, a_i, \vec{w}_i$. Use chain rule.

$$1) \frac{\partial L}{\partial a_0} = \frac{\partial}{\partial a_0} \left[\frac{1}{2} (y - g(a_0))^2 \right] = -(y - g(a_0)) \cdot g'(a_0)$$

$$2) \frac{\partial L}{\partial \vec{w}_0} = \frac{\partial L}{\partial a_0} \cdot \frac{\partial a_0}{\partial \vec{w}_0} = \frac{\partial L}{\partial a_0} \cdot \frac{\partial}{\partial \vec{w}_0} (\vec{w}_0^T \vec{h}) = \frac{\partial L}{\partial a_0} \cdot \vec{h}^T$$

← already computed.

$$3) \frac{\partial L}{\partial a_i} = \frac{\partial L}{\partial a_0} \cdot \frac{\partial a_0}{\partial a_i}$$

← already computed

$$a_0 = \vec{w}_0^T \vec{h} = \sum_{i=0}^m w_{0i} h_i = \sum_{i=0}^m w_{0i} g(a_i)$$

$$\frac{\partial a_0}{\partial a_i} = w_{0i} \cdot g'(a_i)$$

$$\Rightarrow \frac{\partial L}{\partial a_i} = \frac{\partial L}{\partial a_0} \cdot w_{0i} \cdot g'(a_i)$$

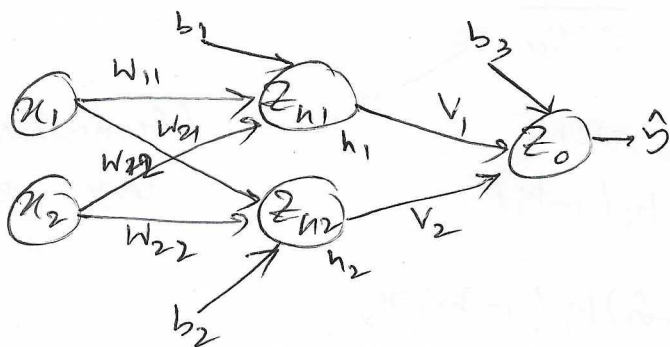
$$4) \frac{\partial L}{\partial \vec{w}_i} = \frac{\partial L}{\partial a_i} \cdot \frac{\partial a_i}{\partial \vec{w}_i}$$

← already computed

$$a_i = \vec{w}_i^T \vec{x} \Rightarrow \frac{\partial a_i}{\partial \vec{w}_i} = \vec{x}^T$$

$$\Rightarrow \frac{\partial L}{\partial \vec{w}_i} = \frac{\partial L}{\partial a_i} \cdot \vec{x}^T$$

(25)

One example of backpropagation

$$L = \frac{1}{2} (y - \hat{y})^2$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$z_{n1} = w_{11}x_1 + w_{12}x_2 + b_1$$

$$z_{n2} = w_{21}x_1 + w_{22}x_2 + b_2$$

$$z_o = v_1 h_1 + v_2 h_2 + b_3$$

$$h_1 = \sigma(z_{n1})$$

$$h_2 = \sigma(z_{n2})$$

$$\hat{y} = \sigma(z_o)$$

Forward pass: $h_i = \sigma(z_{ni})$, $\hat{y} = \sigma(z_o)$

Backpropagation: We'll compute $\frac{\partial L}{\partial v_j}$, $\frac{\partial L}{\partial b_j}$, $\frac{\partial L}{\partial b_3}$, $\frac{\partial L}{\partial w_{jk}}$

$$1) \frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} \left[\frac{1}{2} (y - \hat{y})^2 \right] = -(y - \hat{y}) = \hat{y} - y$$

$$2) \frac{\partial \hat{y}}{\partial z_o} = \sigma'(z_o) \rightarrow \text{We'll write in terms of } \hat{y}$$

$$\text{We have, } \sigma(z) = \frac{1}{1 + e^{-z}} \Rightarrow \frac{d\sigma}{dz} = \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$\text{Now } 1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}$$

$$\text{So, } \sigma(z)(1 - \sigma(z)) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$\text{Hence, } \frac{d\sigma}{dz} = \sigma(z)(1 - \sigma(z))$$

$$\Rightarrow \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\text{So, } \frac{\partial \hat{y}}{\partial z_o} = \sigma'(z_o) = \hat{y}(1 - \hat{y}) \quad (\text{Since } \hat{y} = \sigma(z_o))$$

$$3) \frac{\partial L}{\partial v_j} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_o} \frac{\partial z_o}{\partial v_j}$$

$$\Rightarrow \frac{\partial L}{\partial v_j} = (\hat{y} - y) \hat{y} (1 - \hat{y}) v_j$$

(Gradient of output layer weights)

$$4) \frac{\partial L}{\partial h_j} = \frac{\partial L}{\partial z_o} \frac{\partial z_o}{\partial h_j} = \left(\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_o} \right) \frac{\partial z_o}{\partial h_j} = (\hat{y} - y) \hat{y} (1 - \hat{y}) v_j$$

$$5) \quad \frac{\partial L}{\partial w_{jk}} = \frac{\partial L}{\partial h_j} \frac{\partial h_j}{\partial z_{h_j}} \frac{\partial z_{h_j}}{\partial w_{jk}} \quad \leftarrow x_k$$

already computed $\uparrow$ Similar to step 2 $\sigma'(z_{h_j}) = h_j(1-h_j)$ (Gradient of hidden layer weights)

$$\Rightarrow \frac{\partial L}{\partial w_{jk}} = (\hat{y} - y) \hat{y} (1 - \hat{y}) h_j (1 - h_j) x_k$$

$$6) \quad z_o = v_1 h_1 + v_2 h_2 + b_3$$

$$\Rightarrow \frac{\partial z_o}{\partial b_3} = 1$$

Now $\frac{\partial L}{\partial b_3} = \frac{\partial L}{\partial z_o} \frac{\partial z_o}{\partial b_3}$ (Gradient w.r.t. I/p bias)

$$\Rightarrow \frac{\partial L}{\partial b_3} = (\hat{y} - y) \hat{y} (1 - \hat{y})$$

7) We have, $z_{h_j} = w_{j1} x_1 + w_{j2} x_2 + b_j$

$$\Rightarrow \frac{\partial z_{h_j}}{\partial b_j} = 1$$

Hence, $\frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial z_{h_j}} \frac{\partial z_{h_j}}{\partial b_j} = \frac{\partial L}{\partial z_{h_j}}$

$$\Rightarrow \frac{\partial L}{\partial b_j} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_o} \frac{\partial z_o}{\partial h_j} \frac{\partial h_j}{\partial z_{h_j}}$$

(Gradient of hidden bias)

$$\Rightarrow \frac{\partial L}{\partial b_j} = (\hat{y} - y) \hat{y} (1 - \hat{y}) v_j h_j (1 - h_j)$$